Geometric Brownian Motion

Simulating price paths with the continuous-time workhorse behind Black–Scholes and Monte-Carlo risk engines.

Key Highlights:

- **Geometric Brownian Motion (GBM)** is the canonical continuous-time model for asset prices.
- GBM powers the **Black–Scholes** framework and **Monte-Carlo** risk engines.
- It entails most of the **intuition practitioners** carry about "what could the price do?".
- The framework serves as fundamental bedrock of mathematical finance — advanced applications layer additional features on top of it.

PROJECT CARD

CATEGORY

Quant Foundations & Derivatives

LIBRARIES

NumPy · Pandas · Matplotlib · yfinance · SciPy

ASSETS

SPY (S&P 500 ETF, State Street)

TIMEFRAME

Jan 2018 – Dec 2024

USE

Scenario cones for wealth projections; the engine inside Monte-Carlo pricing & risk systems

```
# Setup

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)  # reproducibility (standard arbitrary number)
plt.rcParams["figure.dpi"] = 110

# ---- house style ------------------------------------------------------
BG    = "#F4F2E8"   # Pale Sisal
INK   = "#151515"   # Anthracite
AQUA  = "#1FFFFF"   # accent
GREY  = "#7A7A7A"

HEAD  = {"fontname": "Switzer", "fontsize": 11, "fontweight": "bold", "color": INK}

plt.rcParams.update({
    "font.family": "monospace",
    "font.monospace": ["Courier Prime", "Courier New", "DejaVu Sans Mono"],
    "font.size": 8,
    "axes.titlesize": 11, "axes.labelsize": 8,
    "xtick.labelsize": 7.5, "ytick.labelsize": 7.5, "legend.fontsize": 7.5,
    "figure.facecolor": BG, "axes.facecolor": BG, "savefig.facecolor": BG,
    "axes.edgecolor": GREY, "axes.labelcolor": GREY,
    "xtick.color": GREY, "ytick.color": GREY,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.3, "grid.color": "#CCCCCC", "grid.linewidth": 0.5,
    "legend.frameon": False,
    "lines.linewidth": 2.0, "lines.markersize": 0,
})
```

1. Introduction

A few simple facts to get up to speed:

- GBM assumes that **percentage returns** — not price changes — are random, normally distributed, and independent over time.
- Prices therefore stay positive and their terminal distribution is **log-normal**.
- **2 parameters** fully describe the model — both estimable directly from historical data:
  - the drift μ (expected annual log-growth plus half-variance)
  - the volatility σ (annualised standard deviation of log returns)

2. Intuition

The logic before diving into equations:

- **Returns compound, prices don't add**
  - A $10 move means something very different at SPY = 100 than at SPY = 600.
  - What is comparable across price levels is the **relative** move.
  - GBM makes randomness proportional to the current price: $dS = \mu S\,dt + \sigma S\,dW$.
- **Why log-normal**
  - If each day multiplies the price by a small random gross return, the log-price is a **sum** of many small independent shocks.
  - Sums of independent shocks are (approximately) normal.
  - Normal log-price ⇒ log-normal price: skewed right, floored at zero.

- **Drift vs. noise**
  - Over short horizons the noise term dominates: $\sigma\sqrt{t}$ shrinks slower than μt as $t \to 0$.
  - Over long horizons drift wins.
  - This is the same $\sqrt{t}$ rule that governs plain Brownian motion — GBM simply wraps it in an exponential.

3. Theory

Below the theoretical foundation and respective equations:

- **The Stochastic Differential Equation (SDE) and its solution**
  - A differential equation with a random term — so it yields a *distribution* of paths, not only one.
  - $\mu S_t\, dt$ is the **drift** (deterministic trend); $\sigma S_t\, dW_t$ is the **diffusion** (the noise).

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

- **Applying Itô's lemma to** $\ln S_t$
  - The $-\frac{1}{2}\sigma^2$ correction comes from $(dW)^2 = dt$.

$$S_t = S_0 \exp\Big[(\mu - \tfrac{1}{2}\sigma^2)t + \sigma W_t\Big]$$

- **Exact discretisation**
  - Because the solution is closed-form, we can simulate **without discretisation error** at any step size Δt.

$$S_{t+\Delta t} = S_t \cdot \exp\Big[(\mu - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\, Z\Big], \qquad Z \sim \mathcal{N}(0,1)$$

The key implementations of those small functions: one draws per-step **gross returns**, the other **compounds** them into price paths.

```
# Implementation

def gbm_returns(mu, sigma, dt, n_steps, n_paths):
    """One-step gross returns under GBM (shape: n_steps x n_paths)."""
    z = np.random.normal(size=(n_steps, n_paths))
    return np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z)

def gbm_paths(s0, mu, sigma, dt, n_steps, n_paths):
    """Price paths of shape (n_steps + 1, n_paths), starting at s0."""
    rets = gbm_returns(mu, sigma, dt, n_steps, n_paths)
    return s0 * np.vstack([np.ones(rets.shape[1]), rets]).cumprod(axis=0)
```

4. Application

- **Calibrate** μ **and** σ **from real data**
  - We pull daily SPY prices, estimate annualised drift and volatility from **log returns**, and let the data drive the simulation.

```
TICKER = "SPY"
START, END = "2018-01-01", "2024-12-31"

import yfinance as yf

px = (yf.download(TICKER, start=START, end=END, auto_adjust=True, progress=False)["Close"]
        .squeeze().rename(TICKER).dropna())
```

```
log_ret = np.log(px / px.shift(1)).dropna()

s0        = float(px.iloc[-1])
sigma_hat = float(log_ret.std() * np.sqrt(252))
mu_hat    = float(log_ret.mean() * 252 + 0.5 * sigma_hat**2)  # drift of the SDE, not of log returns

print(f"{TICKER}: {len(px)} daily observations, last close = {s0:,.2f}")
print(f"mu_hat    = {mu_hat:.2%}  (annualised drift)")
print(f"sigma_hat = {sigma_hat:.2%}  (annualised volatility)")
```

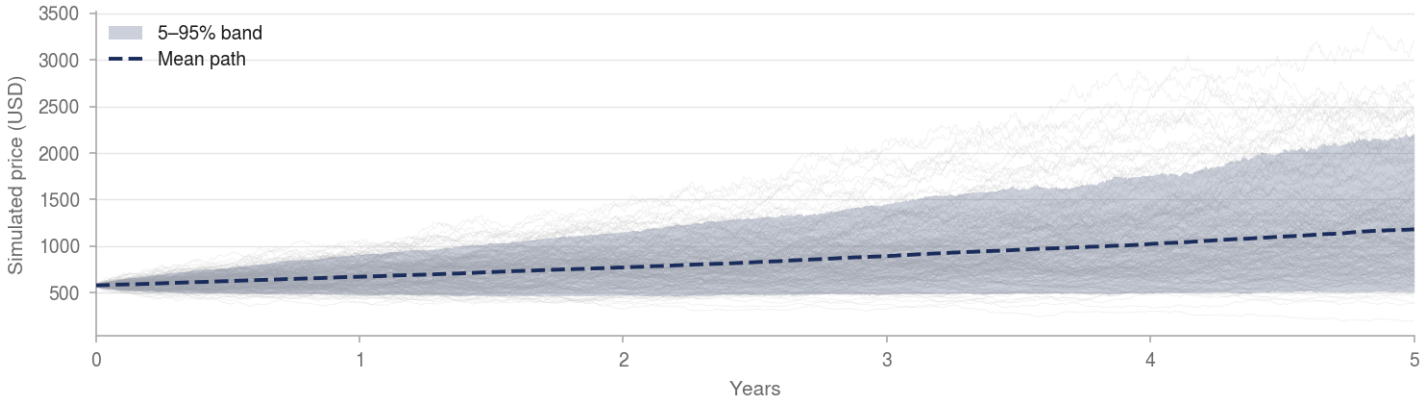
```
SPY: 1760 daily observations, last close = 578.32
mu_hat    = 14.75%  (annualised drift)
sigma_hat = 19.54%  (annualised volatility)
```

- **Simulate 1,000 5-year paths**
  - Grey lines are individual paths, the dashed line is the mean of the simulated distribution, and the band spans the 5th–95th percentile.

```
DT      = 1 / 252
HORIZON = 252 * 5          # 5 years
N_PATHS = 1_000

paths = gbm_paths(s0, mu_hat, sigma_hat, DT, HORIZON, N_PATHS)
t_ax  = np.arange(paths.shape[0]) / 252

p5, p95 = np.percentile(paths, [5, 95], axis=1)

fig, ax = plt.subplots(figsize=(10, 5))
ax.fill_between(t_ax, p5, p95, color=AQUA, alpha=0.35, lw=0, zorder=1,
                label="5\u201395% band")
ax.plot(t_ax, paths[:, :200], color="#BBBBBB", lw=0.35, alpha=0.45, zorder=2)
ax.plot(t_ax, paths.mean(axis=1), color=INK, lw=2.2, ls="--", zorder=3,
        label="Mean path")
ax.set_title(f"{TICKER}: {N_PATHS:,} GBM paths, 5 years  "
             f"(mu={mu_hat:.1%}, sigma={sigma_hat:.1%})", **HEAD)
ax.set_xlabel("Years"); ax.set_ylabel("Simulated price"); ax.legend()
plt.tight_layout(); plt.show()
```

Figure 1. SPY: 1,000 GBM paths, 5 years (mu=14.7%, sigma=19.5%)

Simulated price under geometric Brownian motion, daily steps; the band spans the 5th–95th percentile of 1,000 paths



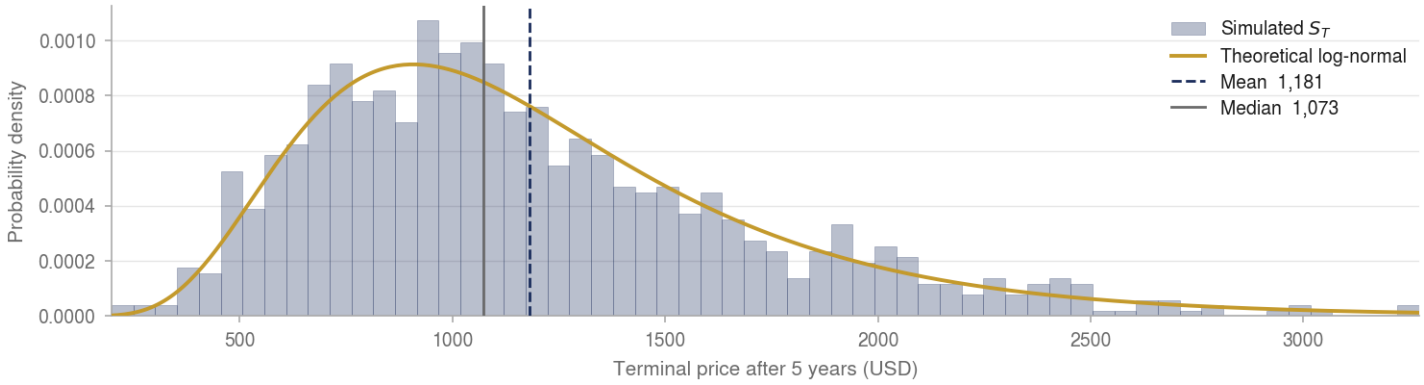
Source: Author calculations; simulated, not actual, results. 1,000 paths from the article's calibrated parameters (S_0 = 578.32, μ = 14.75%, σ = 19.54%), estimated on SPY daily closes, January 2018 – December 2024.

- **Terminal distribution** — is it log-normal?
  - **Verifies the code:** the simulated histogram should match the closed-form density $\ln(S_T/S_0) \sim \mathcal{N}((\mu - \frac{1}{2}\sigma^2)T,\ \sigma^2 T)$ — if it doesn't, the ½σ² term is usually the culprit.
  - **Proves the claim:** section 2 asserts prices end up log-normal; this is where you see the right skew rather than take it on faith.
  - **The practical lesson:** the **mean sits above the median**, so the "average" projected outcome is not the *typical* one — most paths land below it.

```
from scipy import stats

T   = HORIZON / 252
s_T = paths[-1]

x   = np.linspace(s_T.min(), s_T.max(), 400)
pdf = stats.lognorm.pdf(x, s=sigma_hat*np.sqrt(T),
                        scale=s0*np.exp((mu_hat - 0.5*sigma_hat**2)*T))

fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(s_T, bins=60, density=True, color=AQUA, alpha=0.55,
        edgecolor=INK, linewidth=0.25, label="Simulated $S_T$")
ax.plot(x, pdf, color=INK, lw=2.2, label="Theoretical log-normal")
ax.axvline(s_T.mean(),     color=INK, lw=1.8, ls="--", label=f"Mean   {s_T.mean():,.0f}")
ax.axvline(np.median(s_T), color=INK, lw=1.8,          label=f"Median {np.median(s_T):,.0f}")
ax.set_title(f"{TICKER}: terminal price distribution after {T:.0f} years", **HEAD)
ax.set_xlabel("Price"); ax.legend()
plt.tight_layout(); plt.show()
```

Figure 2. SPY: terminal price distribution after 5 years

Simulated terminal prices against the closed-form log-normal density; the mean sits above the median, the signature of the right skew



Source: Author calculations; simulated, not actual, results. Distribution of S_T across the same 1,000 paths, with the closed-form log-normal density overlaid.

- **Sanity check:** switch the drift off
  - With $\mu = 0$ the process becomes a **martingale** — no expected gain — so the mean terminal price must land back at S_0.
  - Zeroing one parameter isolates the rest: this catches a missing $-\frac{1}{2}\sigma^2$ correction that a full simulation might hide.
  - **How to read the result:** the sample mean wobbles around S_0 with standard error $\approx S_0\sqrt{e^{\sigma^2 T} - 1}/\sqrt{N}$. Within ~2 SE is noise; a persistent gap in one direction that *doesn't* shrink as N grows is a bug.

```
paths_nd = gbm_paths(s0, 0.0, sigma_hat, DT, HORIZON, N_PATHS)
print(f"Mean terminal price (no drift): {paths_nd[-1].mean():,.2f}   vs   S0 = {s0:,.2f}")
```

```
Mean terminal price (no drift): 588.06   vs   S0 = 578.32
```

5. Conclusion

Strengths

- **Analytically tractable** — closed-form solutions (Black–Scholes) make it the natural baseline
- **Positive prices, log-normal terminal distribution** — matches the basic stylised fact that prices can't go negative
- **Only two parameters** — both estimable directly from historical log returns
- **Cheap to simulate** — vectorised NumPy generates millions of paths in seconds

Weaknesses & Limitations

- **Constant volatility** — realilty has vol clusters and spikes
- **No jumps** — crashes like March 2020 are far outside its reach
- **Normal log-returns** — real returns have fat tails
- **Independent increments** — momentum and mean-reversion exist

Applications in Practice

- Baseline for option pricing and Monte-Carlo risk engines
- Scenario cones for wealth projections
- Base model against which fancier models must justify their complexity

Alternatives & Extensions

- **Stochastic volatility** — Heston
- **Jump-diffusion** — Merton
- **GARCH-family models** — for clustered volatility